

Quinn Smiley
CIT 381
November 16th, 2025

Project 6

Performance issues:

Both my tables may have issues because they are both referenced in several queries:

1. How many users got a specific result

```
SELECT r.result,  
COUNT(u.username) AS user_count  
FROM results r  
JOIN users u ON r.user_id = u.user_id  
WHERE r.result = 'Princess Mononoke';
```

Indexes Added: CREATE INDEX idx_results_result ON results(result);

CREATE INDEX idx_results_user_id ON results(user_id);

Explain output after indexing: '-> Aggregate: count(u.username) (cost=0.93 rows=1)\n
-> Nested loop inner join (cost=0.7 rows=1)\n -> Filter: (r.user_id is not null)
(cost=0.35 rows=1)\n -> Index lookup on r using idx_results_result (result =
'Princess Mononoke') (cost=0.35 rows=1)\n -> Single-row index lookup on u using
PRIMARY (user_id = r.user_id) (cost=0.35 rows=1)\n'

Summary: Now, it should be able to find all rows in results where the result is 'Princess Mononoke' quickly, and then join those rows to users faster.

2. Number of attempts per user

```
SELECT u.username,  
COUNT(r.attempt_id) AS attempt_count  
FROM users u  
JOIN results r ON u.user_id = r.user_id  
GROUP BY u.username;
```

Indexes used: CREATE INDEX idx_results_user_id ON results(user_id);

```
CREATE INDEX idx_users_username ON users(username);
```

Explain output after indexes: '-> Table scan on <temporary>\n -> Aggregate using temporary table\n -> Nested loop inner join (cost=1.15 rows=2)\n -> Covering index scan on r using idx_results_user_id (cost=0.45 rows=2)\n -> Single-row index lookup on u using PRIMARY (user_id = r.user_id) (cost=0.3 rows=1)\n'

Summary: The joining between tables, count lookups, and filtering by user_id will be faster. In addition, the second index will group without sorting, use the index's ordering, and speed up aggregation.

3. Users with a profile picture

```
SELECT u.username  
FROM users u  
WHERE u.profile_pic IS NOT NULL;
```

Indexes used: CREATE INDEX idx_users_profile_pic_username ON users(profile_pic, username);

Explain output after indexes: '-> Filter: (u.profile_pic is not null) (cost=0.896 rows=3)\n -> Covering index range scan on u using idx_users_profile_pic_username over (NULL < profile_pic) (cost=0.896 rows=3)\n'

Summary: MySQL can retrieve usernames directly from the index.